Intelligent Systems Part B documentation

Vegard Pettersen

February 17, 2025

1 Introduction

The problem, I have chosen for this task, is the Rastrigin function with a variable number of dimensions. For testing and performance metrics, the goal is to get within 0.01 of the real goal in 100 dimensions. To solve this, a continuous genetic algorithm is implemented on the GPU, using Compute shaders in the Unity game engine. In other words, the algorithm is written as a combination of HLSL/GLSL and c#.

2 The problem

Figure 1 shows what the function looks like in 3D. This image is useful to visualize what the algorithm is doing, however the actual problem will be in 100 dimensions or more, which is impossible to visualize. Essentially, a single individual is simply a point in the dimension of the Rastrigin function, and the individual's fitness is what the Rastrigin function outputs for that specific point. The Rastrigin function for N dimensions is as follows:

$$f(\mathbf{x}) = 10N + \sum_{i=1}^{N} \left[x_i^2 - 10\cos(2\pi x_i)\right]$$

Figure 1 is simply this function with a value $N = 2$. For any dimension, the correct solution is simply the dimension's origin point. The goal with this benchmark, is to find the global minima of the function, which is difficult since the function has many local minima that an algorithm can get trapped in. A simpler algorithm, such as newtons method, would most likely never find the correct global minima in 100 dimensions, even given millions of attempts. The correct global minima of the function is simply at the origin point. In other words, when every positional value is equal to zero, the algorithm has found the correct global minima. However, for the Rastrigin function in higher than a few dimensions, finding the actual bottom is practically impossible. The problem is static, as nothing that happens while the algorithm is running can change the shape of the Rastrigin function; The goal is simply to find a point in space. The problem is continuous, as the position can be any real position in space; Well, it is technically discrete, as floats and doubles don't have infinite precision, however it is supposed to represent a continuous problem, and it is practically continuous with 32-bit floats.

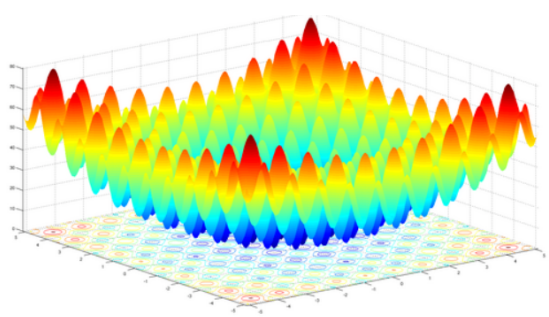

Figure 1: the Rastrigin function in 3D

Lastly, it is Unconstrained, as there are no limits to where the individuals can be, however their start positions are constrained. Explicitly, the problem can be categorized as:

- 1-500 dimensional (100 dimensions in testing)
- static
- continuous
- Unconstrained

3 The Genetic Algorithm

3.1 How it works

The algorithm will first create a population of individuals that have random positions in the range of -5.12 and 5.12 in each axis. The size of this population will remain constant for the entire time the algorithm runs. Each individual will then have a fitness assigned to them, calculated by the Rastrigin function, given their position. After this, a tournament will be held to decide which of these individuals have the best fitness, aka, which have the lowest fitness value. The tournament will hold the same number of rounds as there are individuals, as its goal is to select a set of winners that is as big as the current population, which means that a single individual can be picked multiple times, while another might never win. A round of the tournament consists of selecting four random individuals, and the one with the lowest fitness is the winner. When the winners have been selected, these winners then become parents to new Individuals. For pairs of two and two of the winners, there will be created two children, which are a random combination of the parent's position on each axis, as well as a random offset (mutation) on each axis. Lastly, another tournament will be run to preserve the elites from the previous generation. Ideally, this would simply be to take a percentage of the best from the previous generation, and put them in the place of a percentage of the worst offspring, however, as this is implemented on the GPU, that is not a trivial task. To make something similar to this work on the GPU, a different type of tournament is held. For a round in this tournament, 16 indices next to each other are selected, and from this selection, the fittest individual of the those 16 winners replace the least fit individual of the 16 children. This means that at least the best winner, and a few more, are percerved to the next round, but it is not a perfect solution. Then, lastly, the list of children and a few winners, replace the current population, and the process repeats. Additionally, the mutation scale is defined by a scalar, which multiplies how much the child should be affected by random mutation. This essentially changes how exploitative the algorithm is versus how exploitative it is.

3.2 Performance measure

It is useful to have a way to define performance that reflects how well the algorithm works. Firstly, the algorithm has a threshold oh how close to zero the fitness should get before it considers the problem solved. Additionally, the amount of dimensions should remain constant. For testing performance, the threshold will be at 0.01 in 100 dimensions. The performance can then be measured by how many generations and how much time it took for the algorithm to get closer than 0.01 in 100 dimensions. The amount of generations it takes reflects the efficiency of the algorithm itself, however the amount of generations alone does not directly reflect performance. The time spent, on the other hand, directly reflects its performance. For instance, it may take only 100 generations with 100K individuals, while it may take 1000 generations with 10K individuals. This would indicate that the algorithm is more efficient with more individuals. However, if the amount of time it takes to calculate a single generation with 100K individuals is more than 10 times slower than with 10K individuals, the algorithm is actually faster with 10K individuals despite being less efficient. Since this is running on the GPU, it is complicated to get a concrete measure of performance, as both the GPUs thread count, VRAM speed, GPU-CPU transfer speed, and CPU speed can all individually drastically impact performance, and vary significantly from computer to computer. For this documentation, it was tested on a GPU with roughly 3500 threads, more than enough VRAM, and fast GPU-CPU transfer speed.

3.3 How it is implemented

3.3.1 Compute Shaders

This is implemented on the GPU using compute shaders in the Unity game engine. A compute shader, is essentially a script that runs multiple times at the same time. In the compute shader, the script gets access to a variable, called id.x in this example, which represents which instance of execution it is. For example, if the CPU tells the GPU to execute a compute shader with 500 instances, the first instance will have id.x = 0, while the next instance will have id.x = 1, and both of these instances run at the same time. That way, the compute shader can be programmed to do calculations and memory manipulation in parallel. This is perfect for this issue, since each individual, and each round of a tournament, can be ran at the same time on the GPU as separate instances of the same compute shader.

Additionally, a compute shader can have compute buffers, which is essentially a list of arbitrary length, that is allocated by the CPU, and can be read and written to from the CPU and GPU. A compute buffer is simply a set of addresses in memory, and is represented on both the CPU and GPU. This means that one compute shader can fill a buffer with data, then the CPU can simply tell the next compute shader to use that data without having to actually know what the data contains. For example, the population of individuals will be represented as a compute buffer, which could be multiple megabytes large for high dimensions, and high population counts. This makes it big enough that sending all the data between the GPU and CPU would be very slow. However, with clever implementation, the CPU never has to care what actually is on the buffer. For instance, if there is one compute shader for calculating the individuals' fitness, and another compute shader for running the tournaments, the CPU can simply tell both of them to read from the same addresses of data on the GPU without having to read what those addresses contain.

3.3.2 Implementation

Now that the basics of compute shaders has been explained, the actual implementation can be explained. There are in total five compute shaders:

- to calculate individuals' fitness
- to run the tournament to decide parents
- to breed those parents into a new population of children
- to run the tournament to decide which parents persist to the next generation
- to push the children buffer back into the population buffer

Additionally, there are five compute buffers:

- the positions of the individuals in the population
- the fitness of the individuals in the population
- the positions of the new children
- the fitness of the new children
- the indices of the winners of the previous tournament

These buffers are defined in a separate type of compute shader, which can be included in other compute shaders to ensure all necessary buffers are correctly defined. A compute shader will never run on its own, and a compute buffer will never allocate itself, so all of this must be managed and executed from a separate script that runs on the CPU. The script will first initialize everything that is necessary. An array of random floats between -5.12 and 5.12 is first created, and it represents the positions of all individuals on all axies. This doesn't require being separated by individuals, as it is Blittable, which means the CPU can simply send it to the GPU as raw bytes. Since there is no difference between a random number on one axie or individual versus another, separating by individuals and axies is unnecessary. Then the population buffer will be initialized, and filled with the random floats. The rest

of the buffers are simply allocated without any data, as the GPU will write to these before they are ever read. Finally, to finish initialization, the CPU dispatches the fitness calculator once to initialize all the fitness values. Dispatching the fitness calculator is not necessary after initialization, as the fitness is immediately calculated when a child is created. After initialization, the remaining compute shaders will be executed in order, and the performance will be read back to the CPU and graphed once per frame until the performance is satisfactory. It will happen in the following order:

- the first tournament to decide parents
- breeding to create the next generation of individuals with random mutations
- the second tournament to decide which parents persist to the next generation
- the helper compute shader to push the new generation into the population buffer
- read back the fitness buffer to the CPU
- graph the fittest individual
- decide if the algorithm is finished
- repeat from the start if not finished

For a more detailed explanation of how this is implemented in actual code, the Unity project is included with reasonable file names and code comments.

4 Result and reflection

4.1 Reflection

This algorithm, in many ways, choses brute force computation speed rather than algorithmic efficiency. This is reflected in, for instance, how it roughly selects parents that are likely to be elites, but not guaranteed, while also eliminating children which are likely very bad, but not guaranteed to be. Implementing a more efficient form of elitism would likely be a necessity on a CPU based implementation, however, doing it roughly like this makes the GPU based implementation so much faster that it makes up for the inaccuracy in raw speed. For general purposes, a single GPU core can be considered half as fast as a single CPU core. This generalization, while being very rough and inaccurate, allows us to indicate how much the algorithm gains from being on the GPU. In 100 dimensions with 10000 individuals, the CPU would have to calculate the fitness 10000 times, where each individuals' fitness is roughly 100 calculations. While the GPU in the test computer can do roughly 3000 of these calculations at the same time, meaning it only has to do the fitness calculations 4 times, though doing it half as fast. This equates to the GPU being roughly 1250 times faster in this situation. This is likely quite inaccurate, but when the difference is that large, even a calculation inaccuracy of $\pm 75\%$ would still show the GPU based algorithm being significantly faster. Therefore, taking only this part into account, the same algorithm on the CPU, would spend roughly 12 seconds per generation, while the algorithm running on the GPU can do it 100 times per second. Therefore, it doesn't need to be nearly as optimal to get similar or better performance. In other words, defining population size as pop, and dimension count as dim, the time complexity of the algorithm on the CPU, is $O(pop * dim)$, while on the GPU in the test computer, it is roughly $O(ceil(\frac{pop}{3000}) * dim)$.

4.2 Testing with different values

The algorithm has variables for population size, fitness threshold, and mutation scale. The fitness threshold is unnecessary to change, as the fitness over time can be directly viewed in the graph. Higher population size typically results in faster convergence in terms of generations, however this quickly gets diminishing returns, as the increased time to calculate a single generation exceeds the gains from the extra individuals. With 100 dimensions, anything lower than roughly 2000–2500 individuals will get stuck on local minima, while anything higher than roughly, 10000 individuals will lead to a higher convergence time. On the test computer, the ideal population size seems to be around 3-4000, which makes sense, as this is fairly close to the thread count of the test computer's GPU. Changing mutation

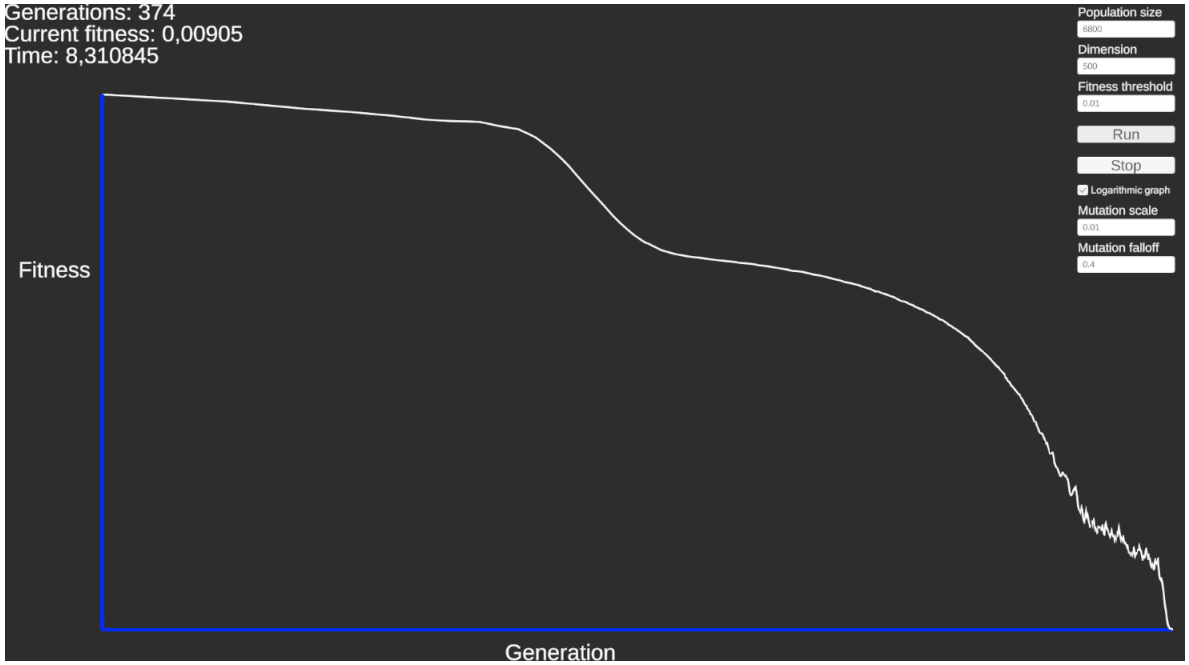

Figure 2: Algorithms performance graphed on a logarithmic scale in 500 dimensions

rate has a large impact on performance. A mutation rate of 0 will mean the algorithm almost always gets stuck before converging close to the goal, while a mutation rate higher than 0.1 makes the algorithm never converge towards anything useful. Typically, the algorithm needs a relatively high mutation rate when far away from the goal, but a low mutation rate when close to it. This was then implemented in code, with a new variable called mutation falloff, which multiplies the mutation rate by $1 - e^{mf*bf}$, where mf is mutation falloff, and bf is the best fitness. However, this makes it often get stuck very close to the goal, and therefore this multiplier is limited to a minimum value of 0.05. Additionally, with this falloff, getting stuck in local minima became more common, therefore the mutation rate is also multiplied by a random value between 0.8 and 1.2, which forces it out of the local minima when necessary. Implementing this change to the behavior of mutation rate, improved the performance for multiple test cases by almost 10x.

4.3 Result

The result of the algorithm is that it works quite well, being able to reach the victory condition in 100 dimensions within, on average, 1 second over 100–200 generations. Additionally, it is able to find the solution in 500 dimensions within 8–12 seconds and 300–900 generations. Figure 2 shows a run in 500 dimensions, where it gets within 0.01 in 8.3 seconds over 374 generations. The fitness over generation is displayed on a logarithmic scale for visibility.

5 Running the code

To run the code, the Unity project can be opened in Unity editor version 6000.0.34f1, or the finished build can be run on a Windows computer that supports DirectX11 or newer, OpenGL 4.3+, or Vulkan. Note that results may be much slower on computers with weak or intigrated graphics cards.